

Tema 2 - Elementos sintácticos del lenguaje

Contenido Teórico

Objetivos

- **Base sintáctica esencial:** escribir comentarios, crear variables y reconocer tipos de datos.
- **Datos y conversiones:** convertir las entradas de texto al tipo adecuado (`int`, `float`, `bool`, `str`).
- **Cadenas:** usar métodos de `str` para normalizar, validar, limpiar, buscar y dividir texto.
- **Operadores:** aplicar operadores aritméticos, de asignación, comparación, lógicos, identidad y pertenencia.
- **Control de flujo:** usar `if/elif/else`, `match/case`, `for`, `while`, `break` y `continue`.
- **Buenas prácticas:** mantener el código legible, indentado y fácil de diagnosticar en soporte técnico.

Comentarios y docstrings

Docstring: cadena multilínea entre triples comillas (Simples o Dobles pero deben abrir y cerrar igual). Se usa habitualmente en la cabecera del código o para documentar módulos, funciones y clases. Permite poner texto con más verborrea.

- **Comentario de línea:** empieza por `#` y no se ejecuta. Sirve para aclarar una instrucción, una decisión o un supuesto. Todo lo que haya a la derecha de una almohadilla se ignora
- Criterio profesional:
 - Explicar qué hace el código sin repetir lo que ya dice el código (comandos, funciones, etc.).
 - Los comentarios deben actualizarse cuando cambia el código. Un comentario desactualizado confunde más que ayuda.

```
'''
    Script de validación de entorno
    (Esto es un Docstring en varias líneas)
'''

# Este print sí se ejecuta
print('Entorno operativo')
# print('No se ejecuta')
```

Variables en Python

Una variable es un nombre que referencia un objeto en memoria.

- **No se declaran:** la variable aparece cuando se asigna un valor.
- **Tipado dinámico:** el tipo está asociado al objeto y puede cambiar en tiempo de ejecución.
- **Case sensitive:** `stock`, `Stock` y `STOCK` son nombres distintos y por lo tanto son variables distintas.
- Los nombres **deben comenzar por letra o guion bajo**. Después pueden contener letras, números y guion bajo.

El [PEP 8 \(Python Enhancement Proposal\)](#) recomienda que en scripts profesionales conviene usar el estilo **snake_case** (todo el nombre de la variable en minúsculas y con guiones bajos) y que se usen nombres descriptivos, ejemplo: `stock_monitores`, `usuario_activo`, `ruta_log`...

En este ejemplo, `stock_portatiles` y `Stock_portatiles` serían variables diferentes.

```
stock_portatiles = 15
Stock_portatiles = 20
print(type(stock_portatiles))
print(stock_portatiles)
print(type(Stock_portatiles))
print(Stock_portatiles)
```

Tipos de datos básicos

Los tipos básicos de datos permiten representar números, texto y valores lógicos.

- **int:** números enteros, positivos o negativos, sin decimales.
- **float:** números reales con decimales. Se usan en cálculos con precisión decimal aproximada.
- **bool:** valores lógicos True o False.
- **str:** cadenas de texto. Se utilizan para mensajes, rutas, nombres, logs, IPs o respuestas de APIs entre otros.
- **type():** función que permite inspeccionar el tipo real de un objeto durante la ejecución.

```
print(type(15))
print(type(25.99))
print(type(True))
print(type('Servidor'))
```

Funciones y métodos

Es importante distinguir ambos conceptos para leer documentación, entender los ejemplos y no confundirse en el momento de escribir código.

- **Función:** se invoca por su nombre y recibe argumentos. **Ejemplos:** `len(mensaje)`, `type(variable)`, `round(numero, 2)`.
- **Método:** pertenece a un objeto y se invoca con punto. **Ejemplos:** `mensaje.upper()`, `texto.strip()`, `ip.split(".")`.

Regla práctica:

- **Si va después de un punto, normalmente es un método del objeto** sobre el que se está trabajando. Los métodos sólo se pueden aplicar a un tipo de objeto. Por ejemplo, `.upper()` a cadenas
- **Si se invoca directamente es una función.** Las funciones son más genéricas y se pueden aplicar a varios tipos de objetos. Por ejemplo `len()` puede medir la longitud de una cadena, una lista o una tupla

```
mensaje = 'servidor activo'
print(len(mensaje))      # función
print(type(mensaje))     # función
print(mensaje.upper())   # método
print(mensaje.split())   # método
```

Conversiones de tipos

Las conversiones adaptan datos de entrada y salida para poder operar correctamente.

- **input():** siempre devuelve texto, aunque el usuario escriba números.
- **int():** convierte una cadena que contenga números sin decimales a entero.
- **float():** convierte una cadena que contenga números con decimales a número real.
- **str():** convierte valores que no sean texto, por ejemplo un entero, a cadena, para concatenar o presentar información.

- **bool():** requiere cautela: cualquier cadena no vacía, incluso la propia "False", se evalúa como True. Sólo la cadena vacía evalúa a False

Si la entrada no es convertible, Python generará `ValueError`. La gestión formal de errores se estudiará más adelante.

```
valor = input('Cantidad: ')
cantidad = int(valor)
precio = float(input('Precio: '))
total = cantidad * precio
print('Total:', round(total, 2))
```

Entrada y salida básica

- **print():** muestra información por pantalla. Puede recibir varios valores de diferentes tipos separados por comas. Por ejemplo una cadena y un entero.
- **sep:** define el separador entre valores en **print()**.
- **end:** define qué se añade al final de la impresión. Por defecto es un salto de línea.
- **input():** solicita texto al usuario desde teclado y siempre devuelve `str`. Si queremos otro tipo de variable hay que hacer la conversión.
- En las prácticas de sintaxis usaremos `input()` para entender conversiones, condiciones y flujos de ejecución.

```
print('Juan', 'Pedro', sep=' | ')
print('Proceso', end='...')
print('OK')
nombre = input('Nombre: ')
print('Hola', nombre)
```

Formateo de Cadenas con f-strings

Las **f-strings** permiten construir cadenas insertando variables o expresiones dentro del texto. Se escriben anteponiendo la letra `f` a la cadena y colocando las expresiones entre llaves `{ }`.

- Son la forma recomendada para generar mensajes legibles cuando se mezclan texto y valores. Evitan concatenaciones largas y reducen la necesidad de convertir manualmente números con `str()`.
- También permiten aplicar formato de salida, por ejemplo limitar decimales, alinear valores o preparar mensajes de log más claros. En scripts de soporte son útiles para mostrar estados, rutas, usuarios, contadores y resultados de operaciones.
- Aunque son muy cómodas, no sustituyen a la validación de datos. Primero se debe asegurar que las variables contienen el valor esperado y después se formatea el mensaje de salida.

```
usuario = "admin"
intentos = 3
precio = 19.995

print(f"Usuario {usuario} bloqueado tras {intentos} intentos")
print(f"Precio final: {precio:.2f} €")
```

Cadenas de texto

Una cadena es una secuencia de caracteres. Se escribe entre comillas simples o dobles.

- Admiten caracteres especiales:
 - `\n`: implica un salto de línea dentro de la cadena de texto.
 - `\\`: carácter para introducir la barra `\` en la cadena de texto.

- `\'`: carácter para introducir una comilla simple en la cadena de texto.
- `\"`: carácter para introducir una comilla doble en la cadena de texto.
- `\t`: carácter para introducir una tabulación horizontal en la cadena de texto.
- **Las cadenas son inmutables:** los métodos no modifican la cadena original, devuelven una nueva cadena que se puede asignar a una variable.
- Se pueden medir, recorrer, dividir y buscar como cualquier otra secuencia.
- En TI las cadenas aparecen constantemente en rutas, logs, nombres de host, direcciones IP, comandos y respuestas de servicios. De ahí su importancia.

```
mensaje = 'Servidor\tACTIVO'
print('Línea 1\nLínea 2')
print(len(mensaje))
print(mensaje.lower())
```

Indexación y Slicing de Cadenas

Las cadenas son secuencias de caracteres y cada carácter tiene una posición. **Python empieza a contar desde 0**, por lo que el primer carácter de una cadena está en el índice **0**. Podemos imaginar una cadena como un array de caracteres.

- También se pueden usar índices negativos. El índice **-1** hace referencia al último carácter, **-2** al penúltimo y así sucesivamente. Esto facilita acceder al final de una cadena sin calcular su longitud.
- El slicing permite obtener fragmentos usando la sintaxis `[inicio:fin]`. El índice de inicio se incluye, pero el índice final no se incluye. Este comportamiento es común en otras secuencias de Python.
- Si se omite el inicio, Python empieza desde el primer carácter. Si se omite el final, toma hasta el último carácter. Opcionalmente puede añadirse un paso con `[inicio:fin:paso]`.

```
mensaje = "Servidor"

print(mensaje[0])      # S
print(mensaje[-1])     # r
print(mensaje[0:3])    # Ser
print(mensaje[:4])     # Serv
print(mensaje[4:])     # idor
```

Métodos de cadenas: transformación

Estos métodos cambian la presentación del texto, pero no alteran el objeto original.

- **capitalize():** pone en mayúscula la primera letra y normaliza el resto.
- **title():** pone en mayúscula el inicio de cada palabra.
- **upper():** convierte todo el texto a mayúsculas.
- **lower():** convierte todo el texto a minúsculas.
- **swapcase():** invierte mayúsculas y minúsculas.

Son útiles, entre otras, para normalizar entradas de usuario antes de compararlas.

```
mensaje = 'servidor ACTIVO'
print(mensaje.capitalize())
print(mensaje.title())
print(mensaje.upper())
print(mensaje.lower())
print(mensaje.swapcase())
```

Métodos de cadenas: comprobación

Los métodos de comprobación devuelven un booleano `True` o `False` y ayudan a prevalidar datos.

- **isalnum():** comprueba si todos los caracteres son alfanuméricos.
- **isalpha():** comprueba si todos los caracteres son letras.
- **isdigit():** comprueba si todos los caracteres son dígitos.
- **islower() / isupper():** comprueban minúsculas o mayúsculas en caracteres alfabéticos.

Espacios, puntos, guiones y otros símbolos pueden hacer que estas comprobaciones devuelvan `False` ya que no se consideran numéricos ni alfabéticos.

```
print('Cisco123'.isalnum())
print('Admin'.isalpha())
print('2026'.isdigit())
print('ROOT_01'.isupper())
print('admin 80'.islower())
```

Métodos de cadenas: limpieza

En procesamiento de texto es frecuente recibir datos con espacios o caracteres sobrantes.

- **lstrip():** elimina caracteres del comienzo de la cadena.
- **rstrip():** elimina caracteres del final de la cadena.
- **strip():** elimina caracteres de ambos extremos.

Por defecto eliminan espacios, tabulaciones y saltos de línea. También pueden recibir como argumento caracteres concretos para que sean eliminados.

No eliminan texto del centro de la cadena, solo actúan sobre los extremos de la cadena.

```
log = '      Error de sistema      '
print(log.lstrip(), end=' [FIN]\n')
print(log.rstrip(), end=' [FIN]\n')
print(log.strip(), end=' [FIN]\n')
print('---LOG---'.strip('-'))
```

Métodos de cadenas: búsqueda y sustitución

Estas operaciones se suelen usar en limpieza de logs, normalización de rutas y análisis de texto simple.

- **replace():** sustituye subcadenas. Puede limitar el número de reemplazos.
- **find():** devuelve la primera posición de una subcadena o -1 si no existe.
- **rfind():** busca desde la derecha y devuelve la última posición encontrada o -1 si no existe.
- **count():** cuenta apariciones de una subcadena.

```
mensaje = 'servidor ACTIVO en la red local'
print(mensaje.replace('e', '3'))
print(mensaje.replace('a', '4', 1))
print(mensaje.find('e'))
print(mensaje.rfind('e'))
print(mensaje.count('a'))
```

Métodos de cadenas: split y join

Estos métodos permiten separar las partes de una cadena con un separador concreto o unir varias cadenas con el tipo de separador indicado.

- **split():** divide una cadena y devuelve una lista de elementos.
 - Sin argumentos, `split()` separa por espacios consecutivos.
 - Con argumento, se usa un separador explícito: punto, coma, dos puntos u otro carácter.
- **join():** realiza la operación inversa: une una secuencia de textos usando un separador indicado.

Ejemplo típico: separar una dirección IP por puntos y reconstruirla después.

```
ip = '192.168.1.50'
partes = ip.split('.')
print(partes)
print(type(partes))
print('.'.join(partes))
```

Funciones generales aplicadas a cadenas

Algunas operaciones no son métodos de **str**, sino funciones generales que aceptan secuencias.

- **len(cadena):** devuelve el número de caracteres.
- **min(cadena):** devuelve el carácter con menor orden según la comparación usada por Python.
- **max(cadena):** devuelve el carácter con mayor orden.

En textos con espacios, números, mayúsculas, minúsculas y símbolos, el resultado puede no coincidir con una ordenación “humana”.

La tabla ASCII se ordena de la siguiente forma: 1º caracteres especiales, 2º números, 3º mayúsculas, 4º minúsculas

```
mensaje = 'servidor ACTIVO'
print(len(mensaje))
print(min(mensaje))
print(max(mensaje))
```

Operadores aritméticos

Permiten realizar cálculos numéricos básicos.

- **+** : suma numérica o concatenación de cadenas.
- **- * /** : resta, multiplicación y división decimal.
- **//** : división entera, descartando decimales.
- **%** : módulo o resto de la división entera.
- ****** : potencia.

La división **/** siempre devuelve **float**, incluso cuando el resultado sea exacto.

```
unidades = 9
cajas = 4
print(unidades + cajas)
print(unidades / cajas)
print(unidades // cajas)
print(unidades % cajas)
print(unidades ** cajas)
```

Operadores de asignación

La asignación permite guardar un valor o actualizar el estado de una variable.

- **=** : asigna una referencia a un objeto o variable.
- **+= -= *= /=** : actualizan una variable usando su valor anterior.
- **%= //= **=** : permiten acumular restos, divisiones enteras o potencias.
- Son útiles para contadores, acumuladores, inventarios, métricas y estados de proceso.

Aunque reducen código, deben usarse solo cuando mejoran la legibilidad.

```
stock = 20
print(stock)
stock += 10
print(stock)
stock *= 10
print(stock)
```

Operadores relacionales y lógicos

Se usan para hacer evaluaciones numéricas y de cadenas

- **Relacionales:** <, >, <=, >=, == y != comparan valores y devuelven booleano.
- **Lógicos:** and, or y not combinan condiciones.
 - **and:** solo devuelve True si todas las condiciones son verdaderas.
 - **or:** devuelve True si al menos una condición es verdadera.
 - **not:** invierte el resultado lógico.

Python permite comparaciones encadenadas. **Por ejemplo:** 0 <= puntuacion <= 100.

```
stock = 30
unidades = 9
print(stock > 0 and unidades == 9)
print(stock < 0 or unidades > 5)
print(not unidades == 5)
print(0 <= 85 <= 100)
```

Identidad y pertenencia

Comprueban la igualdad de referencias a objetos o la pertenencia de un valor a un objeto.

- **Identidad:**
 - **is :** Da verdadero si dos referencias apuntan al mismo objeto en memoria
 - **is not :** Da verdadero si dos referencias no apuntan al mismo objeto en memoria.
 - No deben usarse para comparar valores normales. Para contenido se usa ==.
- **Pertenencia:**
 - **in :** Verdadero si un valor está en una secuencia o colección
 - **not in :** Verdadero si un valor no aparece dentro de una secuencia o colección.
 - La pertenencia funciona con listas, tuplas, conjuntos, diccionarios y cadenas.
 - Diferenciar identidad y contenido evita errores sutiles al trabajar con objetos mutables.

```
a = ['Monitor']
b = ['Monitor']
print(a == b)    # mismo contenido
print(a is b)    # distinto objeto
usuarios = ['Admin', 'Editor']
print('Admin' in usuarios)
```

Valor especial None

None representa la ausencia explícita de valor. **No es lo mismo que 0, False, una cadena vacía o una lista vacía.** Es un valor especial que **indica que todavía no hay dato**, que no se ha obtenido resultado o que una operación no devuelve información útil.

- Aparece con frecuencia al inicializar variables antes de asignarles un resultado real, al trabajar con búsquedas que pueden no encontrar nada o en funciones que no devuelven explícitamente un valor.
- La comprobación recomendada es usar `is None` o `is not None`. En este caso sí tiene sentido usar identidad porque `None` es un objeto único en Python.

Distinguir `None` de otros valores falsos evita errores sutiles. Por ejemplo, una variable puede contener 0 como resultado válido, mientras que `None` indicaría que todavía no se ha calculado nada.

```

resultado = None

if resultado is None:
    print("No hay resultado disponible")

equipo = None
print(equipo is None)

```

Bloques e indentación

Un bloque agrupa instrucciones que pertenecen a una misma estructura.

- Python no usa llaves para delimitar bloques: utiliza la indentación.
- Una indentación incorrecta puede producir `IndentationError` o, peor, cambiar la lógica sin que sea evidente.
- La convención habitual, y como buena práctica, es usar 4 espacios por nivel y no mezclar espacios con tabuladores.
- En soporte técnico, si un código copiado falla, revisar primero espacios, tabuladores y alineación de cada bloque.

```

if servicio_activo:
    print('OK')
    print('Monitorizando')
else:
    print('Revisar servicio')

```

Condicionales if / elif / else

Los condicionales permiten ejecutar código según una condición.

- **if:** evalúa la primera condición.
- **elif:** permite evaluar condiciones alternativas sin anidar excesivamente.
- **else:** cubre el caso restante cuando ninguna condición anterior se cumple.

En código profesional debe priorizarse la legibilidad frente a sintaxis abreviada en una sola línea. También es recomendable evitar anidamientos profundos de condicionales básicos cuando una cadena `if/elif/else` expresa mejor el criterio.

```

if grados < 18:
    print('Temperatura baja')
elif grados > 27:
    print('Sobrecalentamiento')
else:
    print('Estado óptimo')

```

match / case

match/case está disponible desde **Python 3.10** y permite clasificar patrones discretos de cadenas.

- Resulta útil para códigos de prioridad, estados, comandos simples o menús.
- **case 'patrón' :** evalúa si el patrón hace match con la variable evaluada y ejecuta las operaciones
- **case _ :** actúa como caso por defecto cuando no coincide ningún patrón anterior.
- **| :** permite agrupar varias alternativas en el mismo caso.

match/case No sustituye siempre a if: encaja mejor cuando se compara una misma variable contra varios valores conocidos. Por ejemplo un menú de opciones con las opciones asignadas a una variable.

```
match codigo:
    case 'b':
        print('BAJA')
    case 'a' | 'u':
        print('ALTA/URGENTE')
    case _:
        print('No reconocido')
```

Bucle for

El bucle **for** recorre elementos de un iterable.

- Puede recorrer listas, tuplas, cadenas, diccionarios, rangos y otros objetos iterables.
- **for item in componentes:** lee directamente cada elemento.
- **enumerate():** Esta función devuelve índice y valor a la vez, lo que permite tratar el par en cada iteración del bucle. Evitar usar índices cuando basta con recorrer los elementos.
- El bucle **for** es la opción habitual cuando se conoce el conjunto de elementos a procesar.

```
componentes = ['CPU', 'RAM', 'SSD']
for item in componentes:
    print(item)
for indice, nombre in enumerate(componentes):
    print(indice, nombre)
```

range() en bucles for

La función **range()** genera secuencias numéricas para controlar iteraciones.

- **range(fin):** empieza en 0 y llega hasta fin - 1.
- **range(inicio, fin):** empieza en inicio y no incluye fin.
- **range(inicio, fin, paso):** permite saltos positivos o negativos.

Recordatorio importante: el valor final nunca se incluye.

Se usa para contadores, posiciones, índices o recorridos numéricos controlados.

```
for valor in range(10):
    print(valor)
for valor in range(10, 21, 2):
    print(valor)
for valor in range(5, 0, -1):
    print(valor)
```

Bucle while

El bucle **while** repite un bloque mientras una condición sea verdadera.

- Se usa cuando no conocemos de antemano el número exacto de iteraciones.
- La condición debe cambiar durante el bucle, si no cambia, el bucle puede ser infinito.
- Dentro del bucle podemos alterar el comportamiento:
 - **break:** termina el bucle inmediatamente.
 - **continue:** salta el resto de la iteración actual y vuelve a evaluar la condición.
- Todo bucle **while** debe tener una condición de salida clara y verificable.

```
intentos = 0
while intentos < 3:
    clave = input('Código: ')
    if clave == 'admin123':
        print('Acceso concedido')
        break
    intentos += 1
```

break y continue

break y **continue** modifican el flujo normal de un bucle.

- **break:** se usa cuando ya no tiene sentido continuar iterando: acceso concedido, dato encontrado, error crítico.
 - **continue:** se usa para saltar casos que no deben procesarse, pero mantener el bucle activo.
- Deben usarse con moderación. Si aparecen muchas veces, puede ser señal de que el flujo necesita simplificarse.

En scripts de soporte son útiles para validaciones, reintentos y filtrado de elementos.

```
contador = 0
while contador < 5:
    contador += 1
    if contador == 3:
        continue
    print('Paso:', contador)
```

Buenas Prácticas y Errores Frecuentes

Buenas Prácticas:

- **Nombres claros:** `stock_monitores` es mejor que `sm o x`.
- **Indentación consistente:** 4 espacios por nivel y sin mezclar tabs.
- **Conversión explícita:** no asumir que `input()` devuelve números.
- **Bloques con propósito:** cada bloque debe tener un objetivo reconocible.
- **Comentarios útiles:** explicar decisiones, supuestos y límites.
- **Prueba incremental:** ejecutar y observar la salida después de cada cambio.

Errores Frecuentes:

- **TypeError:** intentar combinar tipos incompatibles, por ejemplo texto + número sin convertir.
- **ValueError:** convertir una entrada no válida con `int()` o `float()`.
- **IndentationError:** bloques mal alineados, espacios sobrantes o mezcla de tabs y espacios.
- **NameError:** usar una variable que no existe o escribir mal su nombre.
- **Comparar con is:** usar identidad cuando realmente se quiere comparar contenido con `==`.
- **Bucle infinito:** `while` sin cambio real en la condición de salida.

```
# Conversión explícita
txt = input("Precio: ")
val = float(txt)
if val > 0: # Propósito
    stock = 10 # Nombre claro
    print(f"Total: {val*stock}")
```

Resumen del Tema 2

- Las variables referencian objetos y Python infiere el tipo en tiempo de ejecución.
- `input()` devuelve texto. Las conversiones `int()`, `float()`, `str()` y `bool()` deben usarse con criterio.
- Las cadenas se manipulan principalmente con métodos de `str`, no con funciones independientes.
- Los operadores permiten calcular, comparar, validar identidad y comprobar pertenencia.
- La indentación define bloques. `if`, `match`, `for` y `while` controlan el flujo del programa.
- El siguiente tema profundiza en funciones y organización de código reutilizable.

```
# Siguiente paso
# Encapsular lógica repetida en funciones
```

Flujo de trabajo en el laboratorio

- Trabajarás con los scripts del tema y sus equivalentes en notebooks.
- Desde el repositorio Git podrás descargar el directorio del Tema02.
- Primero ejecuta los scripts completos para observar la salida.
- Después prueba los bloques en notebooks para modificar valores, repetir celdas y aislar conceptos.

El Laboratorio reforzará el conocimiento de los elementos básicos de la sintaxis de Python para la programación estructurada, incluyendo tipos básicos de variables, asignación, conversión y transformación y control de flujo de la ejecución

- La versión de Python usada en los venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python
source .venv/bin/activate
python -version
# Actualizar del repositorio Git
git pull origin main
# Para ejecutar VSCode
code . # usar carpeta Tema02
# Para ejecutar en Terminal
python
```